

ESCUELA POLITÉCNICA NACIONAL

FACULTAD DE INGENIERÍA DE SISTEMAS

**DEVELOPMENT OF STUDIES FOR HAND PROSTHESIS USING
EMG SIGNALS AND ARTIFICIAL INTELLIGENCE**

**ALGORITHM BASED ON DEEP-Q LEARNING (DQN) TO TRAIN AN
AGENT OPERATING A MYOELECTRIC HAND PROSTHESIS
PROTOTYPE**

**TRABAJO DE INTEGRACIÓN CURRICULAR PRESENTADO COMO
REQUISITO PARA LA OBTENCION DEL TÍTULO DE INGENIERO EN
CIENCIAS DE LA COMPUTACIÓN**

DENIS DAVID ARAQUE GUASGUA

denis.araque@epn.edu.ec

DIRECTOR: LORENA ISABEL BARONA LÓPEZ

lorena.barona@epn.edu.ec

DMQ, July 2024

CERTIFICATIONS

I, DENIS DAVID ARAQUE GUASGUA, declare that the curricular integration work described here is my own; that it has not been previously submitted for any degree or professional qualification; and that I have consulted the bibliographic references included in this document.



DENIS DAVID ARAQUE GUASGUA

I certify that the present curricular integration work was developed by DENIS DAVID ARAQUE GUASGUA, under my supervision.

LORENA ISABEL BARONA LÓPEZ
DIRECTOR

COPYRIGHT STATEMENT

Through this declaration, we affirm that the curricular integration work described here, as well as the resulting product(s), are public and will be available to the community through the institutional repository of the Escuela Politécnica Nacional. However, the ownership of the patrimonial rights belongs to the authors who have contributed to the development of this work, observing for this purpose the provisions established by the competent intellectual property authority, internal regulations, and other norms.

DENIS DAVID ARAQUE GUASGUA

LORENA ISABEL BARONA LÓPEZ

DEDICATION

To God, for His guidance and blessings. For every step taken, every door opened, and every lesson learned, I am grateful for His infinite love and for continuing to illuminate my path.

To my mother, Blanca Guasgua, and my father, Jaime Araque. Thank you for being my unconditional pillars, for your love, patience, and sacrifices, which have allowed me to reach this point. Your teachings and unwavering support have been the foundation upon which I have built my future. Through every challenge and achievement, you have always been by my side, providing me with strength and hope.

To Michelle Tapia, for being a loyal companion and constant support throughout my university studies. Your presence has been a source of encouragement and comfort in the most difficult moments, and a joy in the triumphs. Your affection and support have made this journey more bearable and meaningful.

To Juan Fernandez, who has always looked out for my family and provided unconditional support. Thank you for always being willing to help and for offering your generosity and kindness without reservation. Your support has been a blessing and has allowed my family and me to face challenges with greater peace and confidence.

To my brothers, Joe and Mateo. Your encouragement and love have been a source of strength and inspiration. Thank you for always believing in me and pushing me to achieve my best. Your support has meant the world to me and has played a significant role in my success.

ACKNOWLEDGMENT

I want to express my sincerest gratitude to Ivanna Cevallos and Kharol Chicaiza for their friendship and influence in my life. Their constant support and belief in my abilities have inspired me to surpass my own limits and not remain stagnant. Their presence in my life has been a constant driving force for growth and personal improvement.

I would like to express my deepest gratitude to Jenny Torres for the opportunities she has provided me. Her trust in my abilities and her constant support have been fundamental to my professional growth. Thanks to her influence, I have been able to develop skills and gain experiences that have been crucial in my academic and professional journey.

I want to thank the members of The Artificial Intelligence and Computer Vision Research Laboratory “Alan Turing” for opening their doors to me, allowing me to learn from them, and assisting me in the development of this work.

TABLE OF CONTENTS

CERTIFICATIONS	I
COPYRIGHT STATEMENT	II
DEDICATORIA.....	III
ACKNOWLEDGMENT	IV
TABLE OF CONTENTS	V
LIST OF FIGURES.....	VI
LIST OF TABLES	VII
RESUMEN	VIII
ABSTRACT	IX
1 INTRODUCTION	1
1.1 General Objective	2
1.2 Specific Objectives.....	2
1.3 Scope.....	2
1.4 Theoretical Framework	3
2 METHODOLOGY	10
2.1 Data Acquisition	10
2.2 Preprocessing	14
2.3 Reward Function.....	17
2.4 Neural Network and Hyperparameters Configurations.....	21
2.5 Training the Agent.....	23
3 RESULTS, CONCLUSIONS AND RECOMMENDATIONS	24
3.1 Results	24
3.2 Conclusions	31
3.3 Recomendations	32
4 REFERENCES	33

LIST OF FIGURES

Figure 2.1. Overview of the Training Process for the Prosthesis Control Agent...	10
Figure 2.1.1. Data Acquisition Devices.....	11
Figure 2.1.2. Placement of the Myo Armband	11
Figure 2.1.3. Placement of the Flexion Glove.....	12
Figure 2.1.4. Flexion Glove Signals During Flexion Event	12
Figure 2.1.5. Flexion Glove Signals During Extension Event	13
Figure 2.1.6. EMG Signals Captured by Myo Armband During Flexion Event	13
Figure 2.1.7. EMG Signals Captured by Myo Armband During Extension Event .	14
Figure 2.2.1. Flexion Glove with 9 Sensors	15
Figure 2.2.2. Prosthesis with Four Degrees of Freedom	16
Figure 2.2.3. Summation of Flexion Sensors.....	16
Figure 2.3.1. Logic for Defining Correct Action.....	19
Figure 2.3.2. Results of Motor inactivity	20
Figure 2.3.3. Results after correction Reward Function	20
Figure 3.1.1. Impact of k Calibration on Agent Training	25
Figure 3.1.2. Learning curves of different datasets.	25
Figure 3.1.3. Episode graphs.	26
Figure 3.1.4. Training for the complete dataset.	26
Figure 3.1.5. Episode graphs for the complete dataset.	27
Figure 3.1.6. Testing of the trained agent.....	28
Figure 3.1.7. Motor behavior history during testing.	28
Figure 3.1.8. Training after fine-tuning.	29
Figure 3.1.9. Testing of the agent after fine-tuning.....	30
Figure 3.1.10. Testing of the agent on prosthesis.	30

LIST OF TABLES

Table 1.4.1. Key Definitions in Reinforcement Learning.....	4
Table 1.4.2. Key Terms in Neural Network Architectures.....	6
Table 2.4.1. Evaluated Neural Network Configurations.....	21
Table 2.4.2. Hyperparameter Settings for Each Evaluated Configuration	22
Table 3.1.1. Configurations for training in simulation.....	27
Table 3.1.2. Tests with different hyperparameters for fine-tuning.....	29

RESUMEN

Las prótesis mioeléctricas han revolucionado la vida de personas con amputaciones de miembros superiores, aunque aún enfrentan desafíos en control y precisión. Este proyecto aplica técnicas avanzadas de aprendizaje profundo, específicamente Deep-Q Learning (DQL), para mejorar la interpretación de señales electromiográficas (EMG) generadas por los músculos del usuario. La naturaleza variable y ruidosa de las señales EMG complica su interpretación, pero las redes neuronales profundas, permiten al agente aprender y optimizar los movimientos protésicos.

El agente DQL integra señales EMG y datos de posición de los motores de la prótesis para replicar movimientos naturales de la mano. Se utilizarán datos recopilados con una banda EMG y un guante de flexión de un grupo diverso de participantes para asegurar una amplia representación de variaciones en las señales. La implementación del agente implicará desarrollar algoritmos específicos y configurar una interfaz de comunicación efectiva para procesar ambas fuentes de información.

Las pruebas continuas y la evaluación del agente permitirán ajustes iterativos en los parámetros y las funciones de recompensa, optimizando su desempeño en diversas condiciones. Este enfoque promete una mayor precisión y fluidez en el control de las prótesis mioeléctricas, llevando la tecnología a un nuevo nivel. El objetivo final es replicar los movimientos de la mano de una manera tan cercana a una mano biológica como sea posible, transformando significativamente la calidad de vida de los usuarios de prótesis.

PALABRAS CLAVE: aprendizaje profundo, control de prótesis, Deep-Q Learning, prótesis mioeléctrica, redes neuronales, señales EMG.

ABSTRACT

Myoelectric prostheses have revolutionized the lives of people with upper limb amputations, although they still face challenges in control and precision. This project applies advanced deep learning techniques, specifically Deep-Q Learning (DQL), to improve the interpretation of electromyographic (EMG) signals generated by the user's residual muscles. The variable and noisy nature of EMG signals complicates their interpretation, but deep neural networks allow the agent to learn and optimize prosthetic movements.

The DQL agent integrates EMG signals and motor position data from the prosthesis to replicate natural hand movements. Data collected with an EMG armband and a flexion glove from a diverse group of participants will be used to ensure a broad representation of signal variations. The implementation of the agent will involve developing specific algorithms and configuring an effective communication interface to process both sources of information.

Continuous testing and evaluation of the agent will allow for iterative adjustments in the parameters and reward functions, optimizing its performance under various conditions. This approach promises greater accuracy and fluidity in the control of myoelectric prostheses, taking the technology to a new level. The goal is to replicate hand movements as closely to a biological hand as possible, significantly transforming the quality of life for prosthesis users.

KEYWORDS: deep learning, prosthesis control, Deep-Q Learning, myoelectric prosthesis, neural networks, EMG signals.

1 INTRODUCTION

The development of myoelectric prostheses has transformed the quality of life for people with upper limb amputations, although it still faces significant challenges in control precision and efficiency. Imagine a world where prostheses can replicate the natural movements of a hand with a fluidity and accuracy that almost rivals human biology. This is the horizon we aim to reach through the implementation of advanced deep learning techniques, specifically Deep-Q Learning (DQL).

In recent decades, the control of myoelectric prostheses has advanced notably, offering new hope to improve the lives of those who have lost upper limbs. These prostheses, which use electromyographic (EMG) signals generated by the user's residual muscles to control prosthetic movements, offer a promising solution for mimicking the natural movements of the hand. However, the precise and efficient interpretation of these signals remains a challenge due to the inherent complexity and variability of EMG signals.

The use of deep learning algorithms has emerged as a powerful technique to overcome these obstacles. In particular, DQL has proven effective in various control and robotics applications, providing a framework for learning optimal policies from large volumes of data. This approach combines the capability of deep learning with reinforcement learning techniques, allowing the agent to learn through interaction with its environment and the feedback received.

Myoelectric prostheses use EMG signals generated by the muscles remaining after amputation to control prosthetic movements. However, interpreting these signals is complex due to their variable and noisy nature. This is where the power of deep learning comes into play. DQL algorithms combine the robustness of reinforcement learning with the ability of deep neural networks to handle high-dimensional data, enabling the agent to learn from its environment and continuously improve its performance.

The objective of this work is to develop, train, and implement a DQL-based agent to control a right-hand myoelectric prosthesis using EMG signals. To achieve this goal, we will start with a thorough review of the scientific and technical literature on the control of myoelectric prostheses, focusing on research that applies deep learning algorithms such as DQL. This review will provide a solid foundation for our design decisions and ensure that we are using the best practices in the field.

1.1 General Objective

To develop, train, and implement an agent using Deep-Q Learning to control a myoelectric right-hand prosthesis using EMG signals.

1.2 Specific Objectives

1. To review the scientific and technical literature on the control of myoelectric prostheses, focusing on research that applies Deep-Q Learning algorithms.
2. To design an agent using Deep-Q Learning, ensuring the integration of the EMG signal.
3. To train the agent so that the hand prosthesis can replicate opening and closing movements.
4. To evaluate the functioning and performance of the agent through the success rate in executing opening and closing tasks.

1.3 Scope

This work consists of five phases detailed below:

1. Literature Review Phase: In this phase, an exhaustive review of the scientific and technical literature related to the control of myoelectric prostheses will be conducted. The focus will be on previous research on the application of deep learning algorithms, especially Deep-Q Learning, in the operation of hand prostheses. This review will provide a solid foundation for design decisions, ensuring that the agent's approach is supported by the best practices and previous findings in the field.
2. Data Acquisition Phase: In this stage, data collection will be carried out using an EMG armband and a flexion glove simultaneously. The objective is to obtain precise information on hand opening and closing movements, which will be used for both model training and reward function calculation. Data collection will be conducted with a group of 12 people to ensure adequate diversity in the data obtained.
3. Design Phase: During this stage, the comprehensive design of the agent based on the Deep-Q Learning algorithm for the operation of the myoelectric hand prosthesis will be carried out. The focus will be on the simultaneous integration of the electromyographic (EMG) signal with the position data of the prosthesis motors. This design will allow the agent to effectively learn hand opening and closing movements, managing both sources of information and establishing various reward functions.

The prosthesis will have 4 DC motors, each equipped with a magnetic encoder to detect the rotor's angular position, facilitating precise replication of hand movements.

4. **Implementation Phase:** In this phase, the code for training the agent using the Deep-Q Learning algorithm will be implemented. The dual input of the EMG signal and the prosthesis motor position data will be processed through specific algorithms. The communication interface will be configured to transmit the agent's decisions to the prosthesis actuators effectively. The implementation will be iterative, allowing adjustments based on test results.
5. **Testing, Evaluation, and Analysis Phase:** In the final phase, tests will be conducted to evaluate the agent's performance. Additional datasets will be used to verify the agent's ability to replicate hand opening and closing movements under various conditions. Parameters and reward functions will be adjusted to optimize the agent's performance. Additionally, a simulator will be implemented to optimize the agent without affecting the physical prototype of the prosthesis.

1.4 Theoretical Framework

Introduction to Reinforcement Learning

Reinforcement Learning (RL) is a branch of machine learning where an agent learns to make decisions by interacting with an environment to achieve a goal. The agent's objective is to learn a policy that maximizes cumulative future rewards through trial and error. This framework is particularly useful in scenarios where the optimal actions are not known beforehand and must be discovered through exploration [1].

In the context of prosthesis control, RL can be used to train an agent to manage the movements of a prosthetic limb by interpreting signals from devices such as EMG sensors. Deep Reinforcement Learning (DRL), which combines RL with deep neural networks, has been particularly effective in this field due to its ability to handle high-dimensional input spaces and learn complex control policies [2].

Deep-Q Learning

Deep-Q Learning (DQL) is a model-free RL algorithm that approximates the Q-value function using deep neural networks. The Q-value function represents the expected cumulative reward of taking a given action in a given state and following the optimal policy thereafter. The core idea of DQL is to use a neural network to estimate these Q-values,

allowing the agent to learn optimal policies for high-dimensional state spaces [3]. Key definitions in reinforcement learning are presented in Table 1.4.1, which are essential to understand the operation and main components of this approach.

Table 1.4.1. Key Definitions in Reinforcement Learning

Element	Definition
State (s)	The current situation of the agent described by a set of variables that capture the environment's essential aspects [1].
Action (a)	The set of all possible moves the agent can make in a given state [1].
Environment ($\mathcal{E}$)	The external system with which the agent interacts, producing new states and rewards in response to the agent's actions [1].
Reward (r)	The feedback received by the agent after taking an action, indicating the immediate benefit of that action [1].
Policy (π)	The strategy used by the agent to determine its actions based on the current state [2].
Q-value $Q(s, a)$	The expected cumulative reward of taking action a in state s and following the optimal policy thereafter [3].
Experience Replay	A technique where the agent stores its experiences and samples them randomly to break correlation between consecutive updates [3].
Target Network	A secondary neural network used to stabilize the training process by periodically copying the primary network's weights [3].

Data Acquisition and Preprocessing

The initial step in training an RL agent for prosthesis control involves data acquisition. Electromyographic (EMG) signals are collected from devices such as the Myo armband, which capture muscle activity, and flexion gloves, which record hand movements. These raw signals are processed to extract meaningful features that can be used as inputs to the neural network. Feature extraction involves transforming the raw EMG data into a set of features that represent the underlying muscle activity patterns [4],[5].

Normalization of these features is important to ensure consistency across different users and sessions. This process involves scaling the features using statistics such as the mean and standard deviation calculated from a training dataset. Normalization helps in reducing variability and improving the performance of the neural network during training [4],[5].

Electromyography (EMG) data collection is fundamental for controlling robotic prostheses. These signals are non-invasive and provide critical information about muscle activation, which can be used to infer the user's intended movements. Advanced techniques in EMG signal processing allow for the extraction of various features, such as mean absolute value, zero crossings, and waveform length, which are essential for accurate classification and control of prosthetic devices [4].

Normalization is performed to scale the features into a consistent range, typically [0],[1], which is for the stability and performance of machine learning models. This process involves subtracting the mean and dividing by the standard deviation of each feature, calculated from the training data. This standardization helps in mitigating the effects of variability in EMG signals due to differences in electrode placement, muscle fatigue, and other factors [4].

Neural Network Architecture for Prosthesis Control

The neural network architecture used for prosthesis control typically involves deep learning models due to their ability to handle complex, high-dimensional data, and learn intricate control policies. The goal of using these architectures is to develop control strategies that are both intuitive and robust for the user, allowing seamless transitions between different movements and environments.

Deep Neural Networks (DNNs) consist of multiple layers of neurons that transform input data into meaningful outputs through a series of linear and non-linear transformations. In the context of prosthesis control, DNNs are used to process sensory inputs from devices, like EMG sensors, and generate control signals for the prosthetic limb. The architecture typically includes layers such as convolutional layers, recurrent layers (like LSTM), and fully connected layers to capture spatial and temporal dependencies in the data [6].

Different configurations of neural network architectures have been explored to optimize prosthesis control. For example, the use of convolutional neural networks (CNNs) for feature extraction from EMG signals and Long Short-Term Memory (LSTM) networks for capturing temporal dynamics is common. These architectures enable the model to learn from the sequential nature of human motion and make accurate predictions about the necessary control signals [7].

An encoder-decoder structure is often employed in these networks to map high-dimensional input data to a lower-dimensional latent space and then decode it back to the original or another useful form. This structure helps in learning compact and efficient representations of the data, which are critical for robust control. For instance, an encoder might process EMG signals to extract latent features representing muscle activation patterns, which the decoder then uses to generate control commands for the prosthesis [7].

Training these networks involves minimizing a loss function that measures the discrepancy between the predicted and actual control signals. Techniques such as backpropagation and gradient descent are used to update the network weights iteratively. Hyperparameters like learning rate, batch size, and the number of epochs is necessary for the training process. Proper tuning of these hyperparameters is necessary to achieve optimal performance [6].

In practical applications, these DNNs have been shown to significantly improve the control of prosthetic limbs. For instance, in a study involving a powered prosthetic leg, a DNN was used to control various ambulation modes, including level-ground walking and stair climbing. The network achieved high accuracy in predicting the necessary control signals, resulting in smooth and natural transitions between different movements [7]. The following Table 1.4.2 provides definitions for key terms related to neural network architectures, which are fundamental to understanding their structure and functionality.

Table 1.4.2. Key Terms in Neural Network Architectures

Term	Definition
Deep Neural Networks (DNNs)	Neural networks with multiple layers that can model complex relationships in data [6][7].
Convolutional Neural Networks (CNNs)	A type of neural network particularly good at processing spatial data like images [7].
Long Short-Term Memory (LSTM)	A type of recurrent neural network (RNN) that is effective at capturing temporal dependencies [7].
Encoder-Decoder Structure	A neural network architecture that compresses input data into a latent space and then reconstructs it [6][7].
Hyperparameters	Settings like learning rate and batch size that need to be tuned for optimal network performance [6].

Reward Function Design

The reward function in reinforcement learning (RL) plays an important role in guiding the agent's learning process. It provides feedback based on the agent's actions, encouraging behaviors that lead to higher cumulative rewards. In the context of prosthesis control, designing an effective reward function is essential to ensure that the agent learns to perform movements accurately and efficiently.

In reinforcement learning, the reward function quantifies the success or failure of an agent's action in a given state. For prosthetic control, the reward function is designed to promote actions that achieve smooth and natural movements. Typically, it involves a combination of factors such as movement accuracy, energy efficiency, and user comfort [8].

Simulation Environment

Training reinforcement learning agents in a simulation environment is a crucial step in developing effective control strategies for prostheses. The simulation environment provides a safe and controlled setting where agents can learn and refine their policies without the risks and constraints associated with real-world hardware testing. This process involves replicating the physical and dynamic properties of the prosthesis and its interactions within a virtual model.

Simulation environments are essential for RL due to the high sample complexity and extensive exploration required for effective learning. Real-world training of robotic systems can be time-consuming and prone to hardware wear and tear. By leveraging simulations, developers can accelerate the learning process and conduct numerous experiments efficiently [9],[10].

The simulation environment must include representations of the tasks the prosthesis is expected to perform. These tasks could range from simple movements like grasping and lifting objects to more complex activities like walking and stair climbing. Each task is defined by specific state variables and reward functions, guiding the agent to learn the optimal policy for performing these tasks effectively [10].

In a study on deep reinforcement learning for robotic manipulation, researchers demonstrated the effectiveness of simulation environments for training agents to perform tasks such as door opening and object manipulation. The simulation enabled the agent to learn complex manipulation skills through trial and error, significantly reducing the time and resources needed for real-world training [10].

Hardware Training and Real-World Adaptation

Training reinforcement learning (RL) agents in a simulation environment is the initial step; however, transferring these learned policies to real-world hardware is crucial for ensuring practical utility. This phase involves fine-tuning the models to adapt to the physical characteristics and limitations of the prosthetic device, ensuring that the agent performs optimally under real-world conditions.

The transition from simulation to hardware is essential to address discrepancies between the virtual and physical environments. Simulations (included highly detailed simulations) cannot perfectly capture all the nuances of real-world physics, sensor noise, and unmodeled dynamics. Therefore, hardware training is critical to adapt the learned policies to the actual prosthesis, ensuring reliable and accurate performance [11],[12].

The process begins by transferring the trained model from the simulation environment to the hardware. This involves uploading the neural network weights and configuration to the prosthetic device's control system. Initial tests are conducted to observe the performance and identify any significant deviations from expected behavior. Calibration steps are then implemented to adjust for differences between the simulated and real environments. This might involve recalibrating sensors, tuning control parameters, and adjusting the reward function to better reflect the physical constraints and operational range of the prosthesis [11].

Once the initial calibration is complete, the prosthesis undergoes a series of real-world tests. These tests are designed to collect new data on the prosthesis' performance under various conditions and tasks, similar to those encountered in daily life. The agent's policy is then refined based on this new data, using techniques such as online learning or additional supervised learning phases. This iterative process ensures that the prosthesis adapts to real-world dynamics and user-specific requirements [12].

During hardware training, the reward function is often modified to incorporate physical constraints and safety measures. For instance, the encoder positions might be limited to prevent the prosthetic limb from moving beyond safe operational limits. Additionally, new reward terms might be introduced to penalize undesirable behaviors or to encourage specific movements or postures that enhance user comfort and functionality [11].

Evaluation and Validation

The evaluation process typically involves both offline and online metrics to comprehensively assess the performance of EMG-based control systems. Offline metrics include classification accuracy and confusion matrices, which provide a preliminary understanding of the system's ability to distinguish between different movement patterns. However, real-time performance metrics are more indicative of practical usability, as they account for the dynamic nature of prosthetic control in daily activities [13].

Real-time testing involves the user interacting with the prosthesis in a simulated or real-world environment, performing a series of tasks that mimic daily activities. This approach evaluates the system's responsiveness, control fluidity, and overall usability. Metrics such as completion time for tasks, success rate, and user satisfaction are used to gauge the system's practical effectiveness. Real-time tests often reveal insights that offline metrics cannot, such as the impact of delay and the intuitiveness of the control scheme [13].

One of the major challenges in EMG-based control systems is the inherent variability of EMG signals. Factors such as electrode placement, muscle fatigue, and changes in limb position can significantly affect the quality and consistency of EMG signals. Variability due to electrode shift, which occurs when the electrodes move relative to the underlying musculature, can lead to significant decreases in classification accuracy. This issue necessitates frequent recalibration or the use of adaptive algorithms that can adjust to changing signal conditions [13].

Several strategies have been developed to address the variability of EMG signals. These include using more sophisticated signal processing techniques, incorporating additional sensors (e.g., accelerometers) to provide contextual information, and employing machine learning algorithms capable of adapting to new conditions. For instance, incorporating data from multiple limb positions and force levels during training can enhance the system's robustness to signal variability. Additionally, adaptive algorithms that continuously update the model based on new data can help maintain high performance despite changing conditions [13].

2 METHODOLOGY

In this section, we detail the steps to train the agent using Deep-Q Learning. Figure 2.1 provides an overview of these steps, starting with data acquisition, followed by data processing, the reward function definition, the neural network configuration, and the training of the agent. The training process begins in a simulation environment, allowing for multiple iterations and adjustments without physical risks. Once the agent has been adequately trained in the simulation, a fine-tuning process is carried out on the hardware to adapt the model to the real-world conditions of the prosthesis. As a result, this comprehensive process yields an agent capable of sending the correct actions to the motors of the prosthesis, effectively responding to the hand movements generated by a person using the Myo armband.

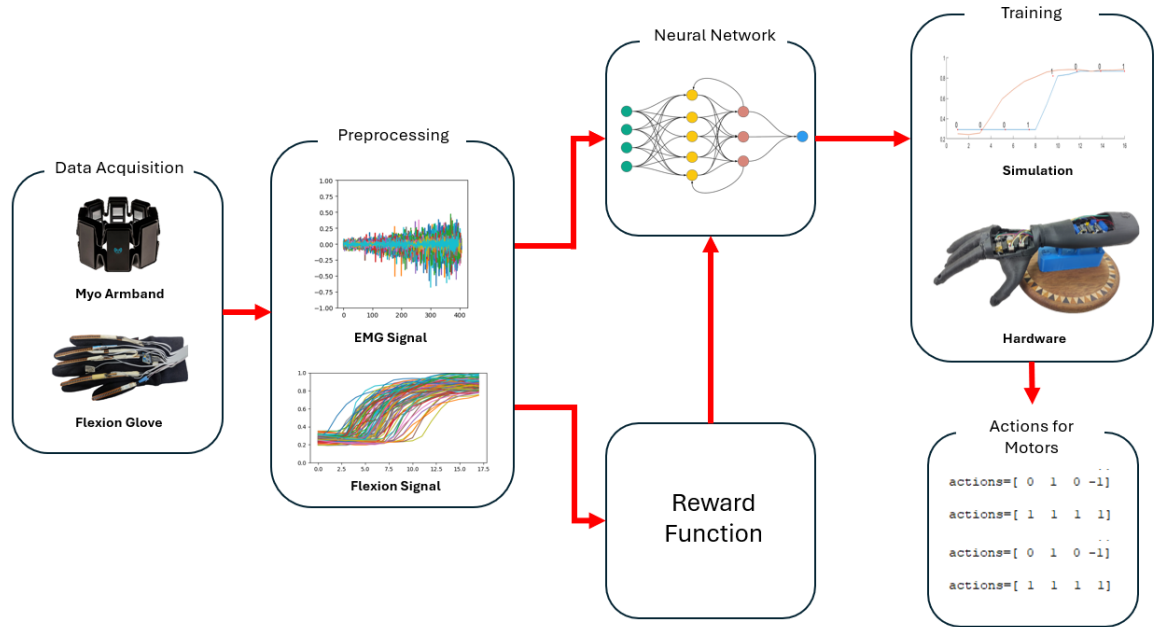


Figure 2.1. Overview of the Training Process for the Prosthesis Control Agent

2.1 Data Acquisition

For data acquisition, two events were defined: flexion (hand closing) and extension (hand opening). These events were recorded with durations of 2, 3, and 5 seconds each, in order to capture data at different speeds of hand closing and opening. The devices used were the Myo armband, which captures EMG signals, and a flexion glove, which records the flexion signal (Figure 2.1.1).



a) Myo Armband



b) Flexion Glove

Figure 2.1.1. Data Acquisition Devices

Data were collected from a group of 12 individuals, obtaining 100 samples for each event and duration, resulting in a total of 600 samples per person. This formed a dataset of 7200 samples in total. As mentioned in the previous section, one of the main challenges in controlling a myoelectric prosthesis is the variability of the signals due to various factors, such as the position of the Myo armband. To mitigate this variability, rigorous controls were established during the placement of the armband. Participants were instructed to stretch their arm with the Myo logo facing upwards, as shown in Figure 2.1.2, and then the flexion glove was placed, as shown in Figure 2.1.3.



Figure 2.1.2. Placement of the Myo Armband



Figure 2.1.3. Placement of the Flexion Glove

The signals from the flexion glove increase during the flexion event (Figure 2.1.4) and decrease during the extension event (Figure 2.1.5).

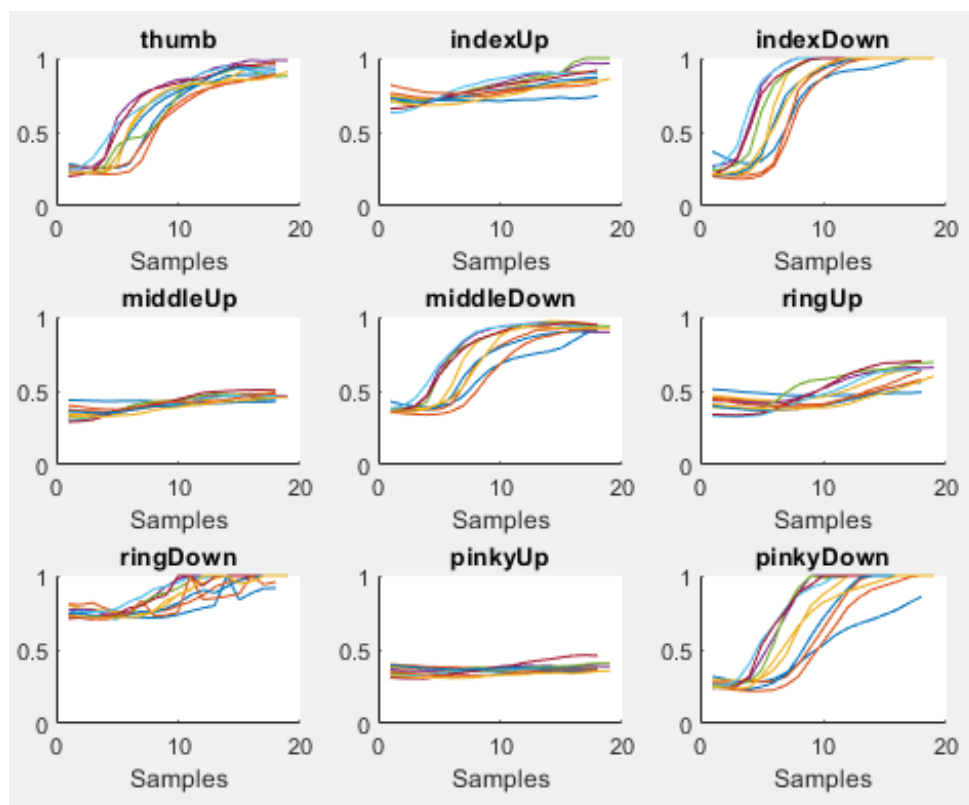


Figure 2.1.4. Flexion Glove Signals During Flexion Event

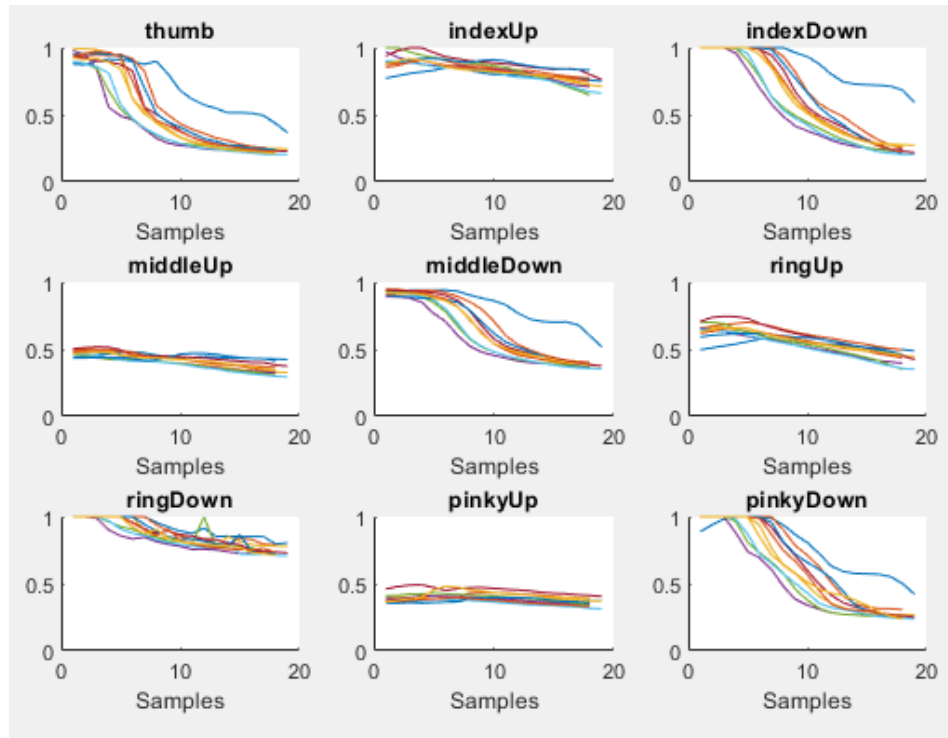


Figure 2.1.5. Flexion Glove Signals During Extension Event

The EMG signals captured by the Myo armband during the flexion event are shown in Figure 2.1.6, and during the extension event in Figure 2.1.7.

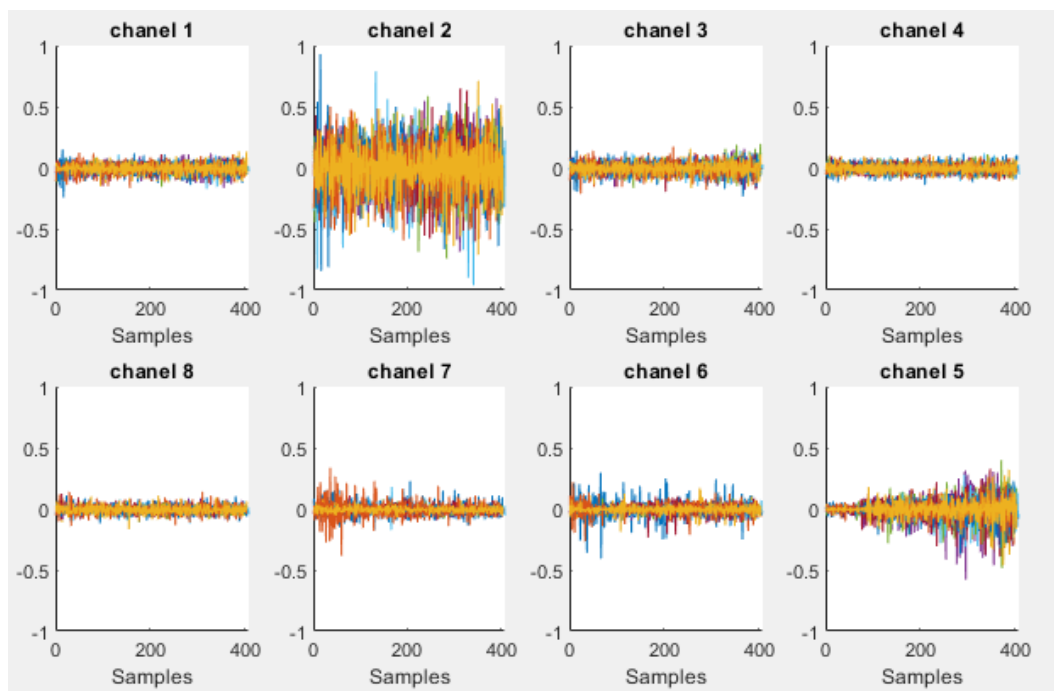


Figure 2.1.6. EMG Signals Captured by Myo Armband During Flexion Event

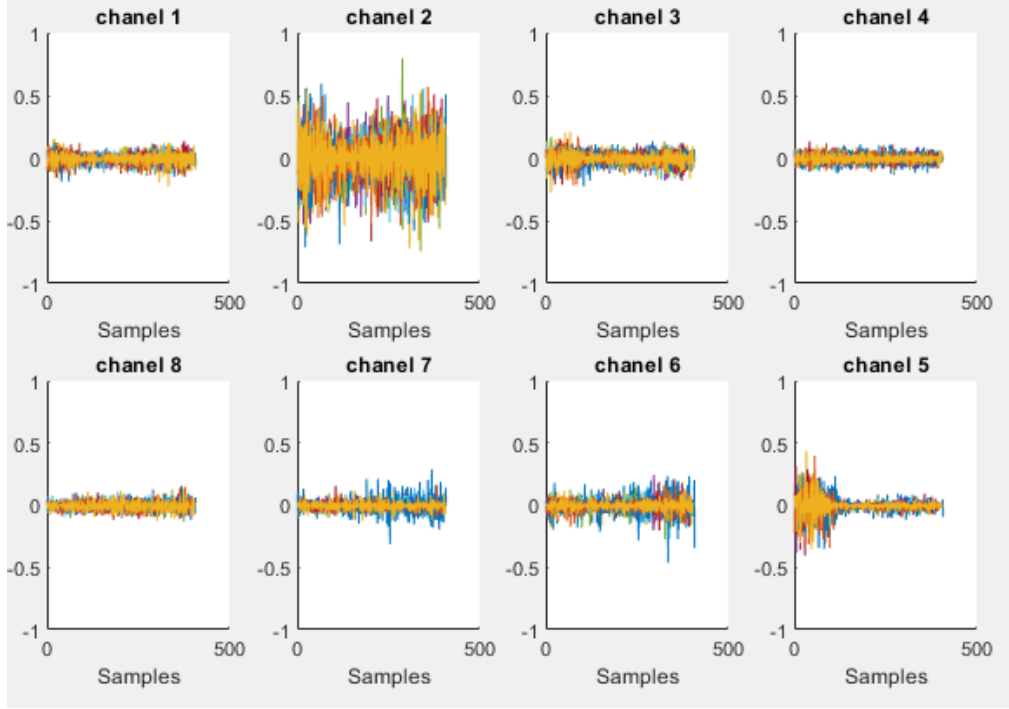


Figure 2.1.7. EMG Signals Captured by Myo Armband During Extension Event

2.2 Preprocessing

The raw EMG signal is represented as a matrix $E \in \mathbb{R}^{N \times 8}$, where N is the number of samples and 8 is the number of channels of the Myo armband. The raw EMG signal undergoes a feature extraction process to obtain relevant information, as shown in Equation 2.2.1. This process transforms the signal E into a feature vector $f_{EMG} \in \mathbb{R}^{F \times 1}$, where F is the number of extracted features. In this case, a total of 40 features have been extracted.

$$f_{EMG} = \text{featureCalculator}(E)$$

Equation 2.2.1. EMG Feature Extraction Process

The extracted features are normalized to ensure they are within an appropriate range for use in the neural network. This normalization is performed using predefined values C and S , which play a crucial role in the process.

Where:

C : Represents the mean values of the features, calculated from a previous training set.

S : Represents the standard deviations of the features, also calculated from the previous training set.

Normalization is performed for each individual feature, f_i , as shown in Equation 2.2.2.

$$f_{EMG, norm_i} = \frac{f_{EMG_i} - C_i}{S_i}$$

Equation 2.2.2. Feature Normalization Process

Where:

f_{EMG_i} it is the value of the i -th feature.

C_i it is the mean of the i -th feature in the training set.

S_i it is the standard deviation of the i -th feature in the training set.

The data from the glove were captured using flexion sensors and are represented as a matrix $G \in \mathbb{R}^{N \times 9}$, where 9 is the number of flexion sensors in the glove, as shown in Figure 2.2.1.

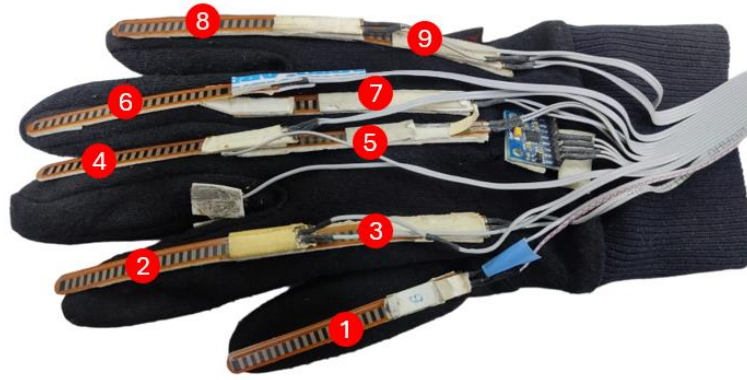


Figure 2.2.1. Flexion Glove with 9 Sensors

The values from the flexion sensors corresponding to each finger are summed to reduce the dimension of G to $G_{red} \in \mathbb{R}^{N \times 4}$, as shown in Equation 2.2.3.

$$G_{red}(i) = \sum_{j \in \text{sensors of finger } i} G(:, j)$$

Equation 2.2.3. Dimensionality Reduction of Flexion Sensor Data

This process allows the glove data to be comparable with the motor data. It is important to note that, for this procedure, given the 4 degrees of freedom (number of motors in the prosthesis) as shown in Figure 2.2.2, the values from flexion sensors of the ring and little fingers have been summed (named as reduced glove data), as illustrated in Figure 2.2.3.

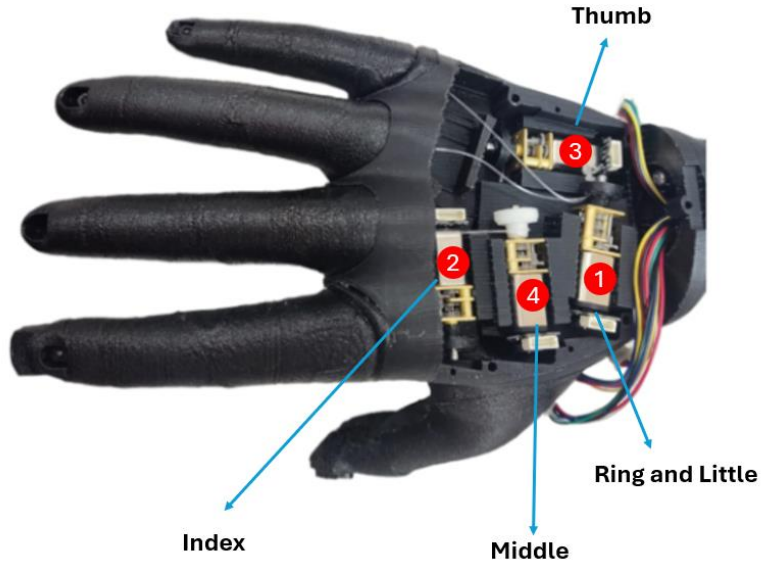


Figure 2.2.2. Prosthesis with Four Degrees of Freedom



Figure 2.2.3. Summation of Flexion Sensors

The reduced glove data are normalized to ensure they are within the range $[0, 1]$. This normalization is performed by dividing the sensor values by their respective maximums L_{max} , as shown in Equation 2.2.4.

$$G_{norm} = \frac{G_{red}}{L_{max}}$$

Equation 2.2.4. Normalization of Reduced Glove Data

Where $L_{max} = [MaxRingLittle, MaxIndex, MaxThumb, MaxMiddle]$, resulting in $L_{max} = [4092, 2046, 1023, 2046]$, which are the maximum values of the flexion sensor after dimensionality reduction. The motor data are captured from the prosthesis encoders and are represented as a matrix $M \in \mathbb{R}^{N \times 4}$.

The motor data are normalized using the encoder limits to scale the data to the range [0,1]. This normalization is performed by dividing the encoder values by their respective maximums L_{enc} , as shown in Equation 2.2.5.

$$M_{norm} = \frac{M}{L_{enc}}$$

Equation 2.2.5. Normalization of Motor Data

Where $L_{enc} = [26500, 11500, 8500, 9000]$ are the maximum values of the encoders.

The normalized EMG data $f_{EMG,norm}$ and the normalized motor data M_{norm} are combined to form the current state of the environment, which is used as input to the neural network, as shown in Equation 2.2.6.

$$s = \begin{bmatrix} f_{EMG,norm} \\ M_{norm} \end{bmatrix}$$

Equation 2.2.6. Formation of the Current State

2.3 Reward Function

The reward function is used to evaluate the actions of an agent in the context of controlling a myoelectric prosthesis. The reward is based on several factors, including the direction of movement, the correctness of the action, the distance between the current position and the desired position, and the penalty for inactivity. The structure of the reward function is detailed and explained as follows:

Several parameters used in the reward function were defined:

- ***k***: Factor that balances the penalty based on distance.
- ***dirInverse***: Penalty for moving in the wrong direction.
- ***wrongStop***: Penalty for stopping when it should be moving.
- ***goodMove***: Reward for moving in the correct action.
- ***goodMove2***: Reward for stopping correctly.
- ***inactivityPenalty***: Penalty for inactivity.
- ***moveIncentive***: Incentive for any movement.

For data reading, the reduced and normalized flexion positions G_{norm} , are used, along with the motor data based on the encoders, which must also be normalized M_{norm} , as shown in Equations 2.3.1 and 2.3.2.

$$flexConv_i = M_{norm}$$

Equation 2.3.1. Encoder values for Motor i

$$posFlex_i = G_{norm}$$

Equation 2.3.2. Flexion values for Finger i

flexConv represents the current position of the motors.

posFlex represents the desired flexion position based on the glove data.

A persistent variable *previousPosFlex*, is used to store the previous flexion position for each motor. If it is the first time the function is executed, it is initialized to zero, as shown in Equation 2.3.3.

$$previousPosFlex_i = 0, \forall i$$

Equation 2.3.3. Initialization of Previous Flexion Position

For each motor i , the correct action is determined based on the previous flexion position (*previousPosFlex_i*) and the current position (*posFlex_i*):

$$correctAction_i = \begin{cases} 1, & \text{if } previousPosFlex_i < posFlex_i \\ -1, & \text{if } previousPosFlex_i > posFlex_i \\ 0, & \text{if } previousPosFlex_i = posFlex_i \end{cases}$$

The actions that the motors (*action_i*) can take correspond to:

- 1: Forward
- 0: Immobile
- -1: Backward

The logic for defining *correctAction* for the events of Flexion (Hand Closing) and Extension (Hand Opening) is shown in Figure 2.3.1.

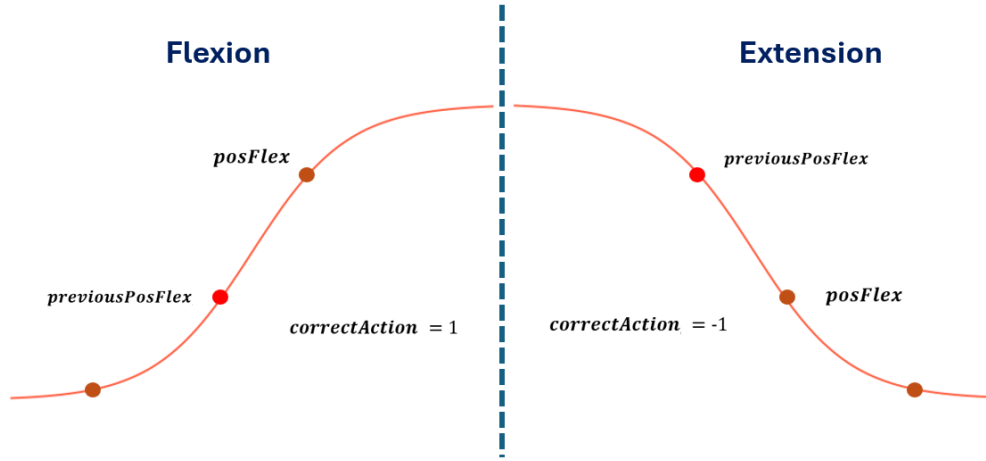


Figure 2.3.1. Logic for Defining Correct Action

The reward or penalty is assigned based on the action taken $\mathbf{action}_i$ compared to the correct action $\mathbf{correctAction}_i$:

$$\mathbf{rewardVector}_i = \begin{cases} \text{goodMove}, & \text{if } \mathbf{action}_i = \mathbf{correctAction}_i \wedge \mathbf{action}_i \neq 0 \\ \text{goodMove2}, & \text{if } \mathbf{action}_i = \mathbf{correctAction}_i \wedge \mathbf{action}_i = 0 \\ \text{wrongStop}, & \text{if } \mathbf{action}_i = 0 \text{ y } \mathbf{correctAction}_i \neq 0 \\ \text{dirInverse}, & \text{if } \mathbf{action}_i \neq 0 \text{ y } \mathbf{action}_i \neq \mathbf{correctAction}_i \end{cases}$$

An incentive is added if the actions are different from 0, as shown in Equation 2.3.4.

$$\mathbf{rewardVector}_i = \mathbf{rewardVector}_i + \text{moveIncentive}$$

Equation 2.3.4. Incentive for Non-Zero Actions

A penalty is calculated based on the distance between $\mathbf{posFlex}$ and $\mathbf{flexConv}$, as shown in Equation 2.3.5.

$$\mathbf{distance} = |\mathbf{posFlex} - \mathbf{flexConv}|$$

Equation 2.3.5. Absolute value of the calculated distance

The reward vector $\mathbf{rewardVector}$ is decreased by the product of the balance factor k and the $\mathbf{distance}$, as shown in Equation 2.3.6.

$$\mathbf{rewardVector}_i = \mathbf{rewardVector}_i - k \cdot \mathbf{distance}$$

Equation 2.3.6. Adjustment of Reward Vector Based on Distance Penalty

Finally, the final reward is calculated as the average of *rewardVector*, as shown in Equation 2.3.7.

$$\mathbf{reward} = \frac{1}{n} \sum_{i=1}^n \mathbf{rewardVector}_i$$

Equation 2.3.7. Calculation of the Final Reward

The reward function was adjusted and modified to enhance the agent's learning outcomes. One of the main issues encountered was motor inactivity (Figure 2.3.1). The agent learned that keeping all motors inactive by setting $\mathbf{action}_i = \mathbf{0}$ was the best strategy to accumulate rewards over time. To counteract the motor inactivity and encourage a more dynamic behavior from the agent, the *moveIncentive* reward was introduced.

Additionally, the *goodMove2* reward was added to cover cases when $\mathbf{correctAction}_i = \mathbf{0}$. This was done to reduce the reward value when the correct action is to remain stationary ($\mathbf{action}_i = \mathbf{0}$). By doing so, inactivity errors were corrected, encouraging the agent to take actions that optimize the movement and control of the prosthesis (Figure 2.3.2).

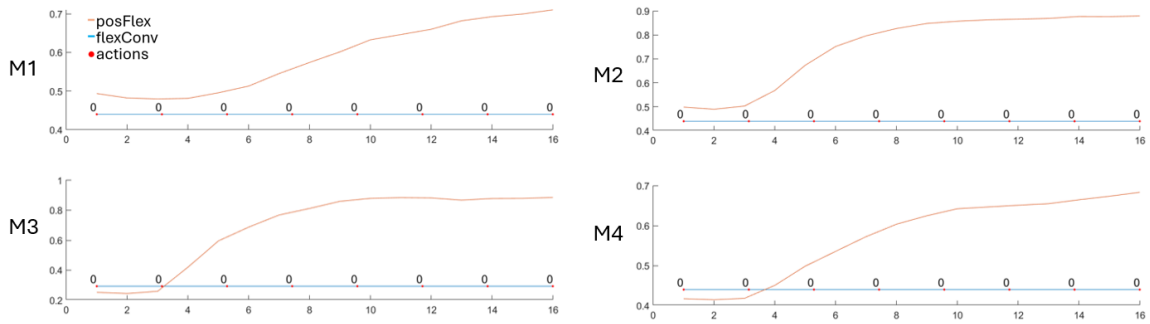


Figure 2.3.2. Results of Motor inactivity

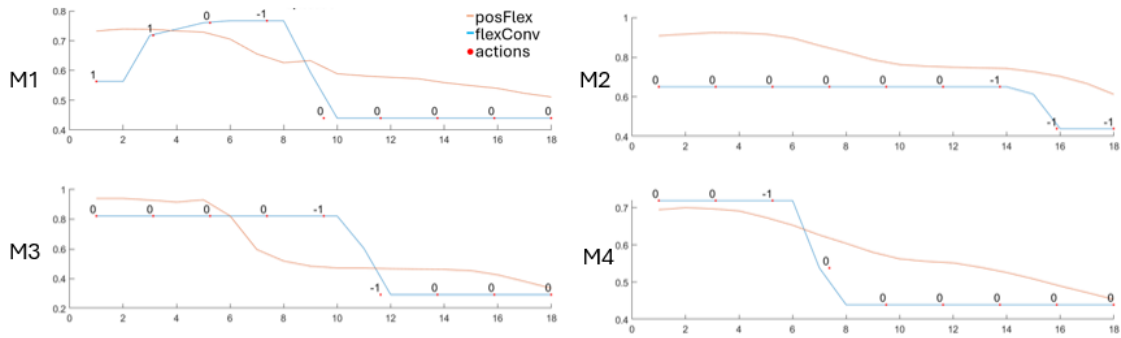


Figure 2.3.3. Results after correction Reward Function

2.4 Neural Network and Hyperparameters Configurations

This section describes the network architectures considered and the training parameters used to evaluate the performance of the myoelectric prosthesis controlled by the agent. The evaluated configurations are detailed below, along with the additional experiments conducted to better understand the impact of varying these parameters. The neural network architecture was varied in terms of the number of layers, unit hidden per layer, and the activation function used. Table 2.4.1 shows the different configurations evaluated.

Table 2.4.1. Evaluated Neural Network Configurations

Configuration	Number of Hidden Layers	Units per Hidden Layer	Activation Function
A	2	14,14	ReLu
B	2	64,32	ReLu
C	2	64,64	ReLu
D	3	28,14,28	ReLu
E	3	28,28,56	ReLu
F	3	64,64,32	ReLu
G	3	128,64,32	ReLu
H	4	28,14,14,28	ReLu
I	4	32,32,16,8	ReLu
J	4	128,64,32,16	ReLu
K	4	256,128,64,32	ReLu
L	5	128,64,32,64,128	ReLu
M	5	256,128,64,128,256	ReLu
N	5	64,32,32,16,8	ReLu
O	5	32,32,16,16,8	ReLu

Different hyperparameter configurations were used for training the neural network. Adjustments were made based on the results obtained after training, aiming to ensure efficient learning and consistent outcomes. Table 2.4.2 summarizes the hyperparameter settings for each evaluated configuration.

Table 2.4.2. Hyperparameter Settings for Each Evaluated Configuration

α	λ	β	μ	γ	τ	ν	ϵ_{decay}	ϵ_{min}
1E-03	1E-04	1024	10000	0.99	1	1E-03	1E-03	0.01
1E-04	1E-05	512	20000	0.99	1	1E-02	1E-04	0.01
5E-04	1E-03	2048	15000	0.95	2	1E-03	1E-03	0.15
1E-03	1E-04	1024	10000	0.99	1	5E-03	1E-04	0.01
1E-05	1E-06	512	5000	0.9	2	1E-04	1E-03	0.02
5E-05	1E-04	256	15000	0.95	1	1E-03	1E-03	0.01
1E-03	1E-03	512	20000	0.9	1	5E-04	1E-03	0.02
5E-04	1E-02	2048	10000	0.99	2	1E-03	1E-03	0.01
1E-04	1E-04	1024	15000	0.95	1	5E-03	1E-03	0.01

Where:

α : The learning rate used to adjust the model's weights during training.

λ : The L2 regularization coefficient used to prevent overfitting by penalizing large weight values.

β : The mini-batch size used during training to compute the gradient.

μ : The experience buffer length that stores transition for experience replay learning.

γ : The discount factor that determines the importance of future rewards compared to immediate rewards.

τ : The frequency at which the targets are updated in deep learning algorithms.

ν : The target smoothing factor used to softly update the target parameters.

ϵ_{decay} : The epsilon decay used to explore the action space.

ϵ_{min} : The minimum epsilon value to ensure there is always some exploration.

2.5 Training the Agent

The training of the hand prosthesis control agent is initially carried out in a simulation environment. This process is based on reinforcement learning techniques, where the agents learn to perform specific tasks through interactions with the simulated environment. The simulation training can be described by the following process:

- The simulated environment, $\mathcal{E}$, provides observations $\mathbf{s}_t$, to the agent at each time step t .
- The agent, $\mathcal{A}$, takes an action $\mathbf{a}_t$ based on the current observations $\mathbf{s}_t$.
- The agent receives a reward r_t from the environment after taking an action. The reward measures how well the agent has performed the desired task.
- The agent's goal is to learn a policy $\pi(\mathbf{a}|\mathbf{s})$ that maximizes the accumulated reward over time.
- The policy is updated using reinforcement learning algorithms, such as Deep-Q Learning (DQN).

The expected value of the accumulated reward, known as the return, is defined as shown in Equation 2.5.1.

$$R_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

Equation 2.5.1. Definition of the Expected Value of the Accumulated Reward

The agent explores the environment, collects experiences $(\mathbf{s}_t, \mathbf{a}_t, r_t, \mathbf{s}_{t+1})$, and iteratively updates its policy π . The action-value function $Q(\mathbf{s}, \mathbf{a})$ is updated using the Bellman equation, as shown in Equation 2.5.2.

$$Q(\mathbf{s}_t, \mathbf{a}_t) = Q(\mathbf{s}_t, \mathbf{a}_t) + \alpha[r_t + \gamma \max_a Q(\mathbf{s}_{t+1}, a) - Q(\mathbf{s}_t, \mathbf{a}_t)]$$

Equation 2.5.2. Bellman equation

In the simulation, the environment provides a virtual representation of the prosthesis and its interactions. The agent receives states from the simulated environment and decides on actions that control the prosthesis. The simulation allows for conducting multiple experiments with different control policies without the risks associated with direct hardware use.

Once the agent has been trained in the simulated environment, fine-tuning is performed on the hardware. This step is crucial for adapting the learned model to the physical realities of the prosthesis to ensure its proper functioning in the real world. The following steps outline the fine-tuning process:

- The model trained in simulation is transferred to the prosthesis hardware.
- The model is retrained to adjust for discrepancies between the simulation and the real hardware.
- The agent undergoes a series of tests in the real environment, where new experiences are collected.
- These experiences are used to refine the model, adjusting parameters and recalibrating the control policy.
- The reward function is adapted to consider the physical constraints and characteristics of the prosthesis (such as setting limits on encoder positions to prevent damage to the prosthesis).
- The agent's performance is validated in specific tasks and optimized to improve accuracy.
- The reward function may include additional terms to penalize undesired behaviors or to favor certain postures or movements.

Training process with the real prosthesis aims to adjust the models to real-world conditions. Real-time feedback and the agent's ability to adapt are crucial for ensuring optimal performance.

3 RESULTS, CONCLUSIONS AND RECOMMENDATIONS

3.1 Results

In this section, the results obtained through various tests conducted to determine the best architecture configurations and hyperparameters are presented. The training process is shown from software simulation to fine-tuning in hardware, highlighting the changes and improvements implemented.

Initially, a calibration of the factor that balances the penalty based on distance (k) was carried out. The variation of this value has a significant impact on training because a small value can result in an insignificant penalty, while a value that is too large could

underestimate the correct actions previously taken. Therefore, it is important to find the optimal value to not affect the proper learning of the agent, as observed in Figure 3.1.1.

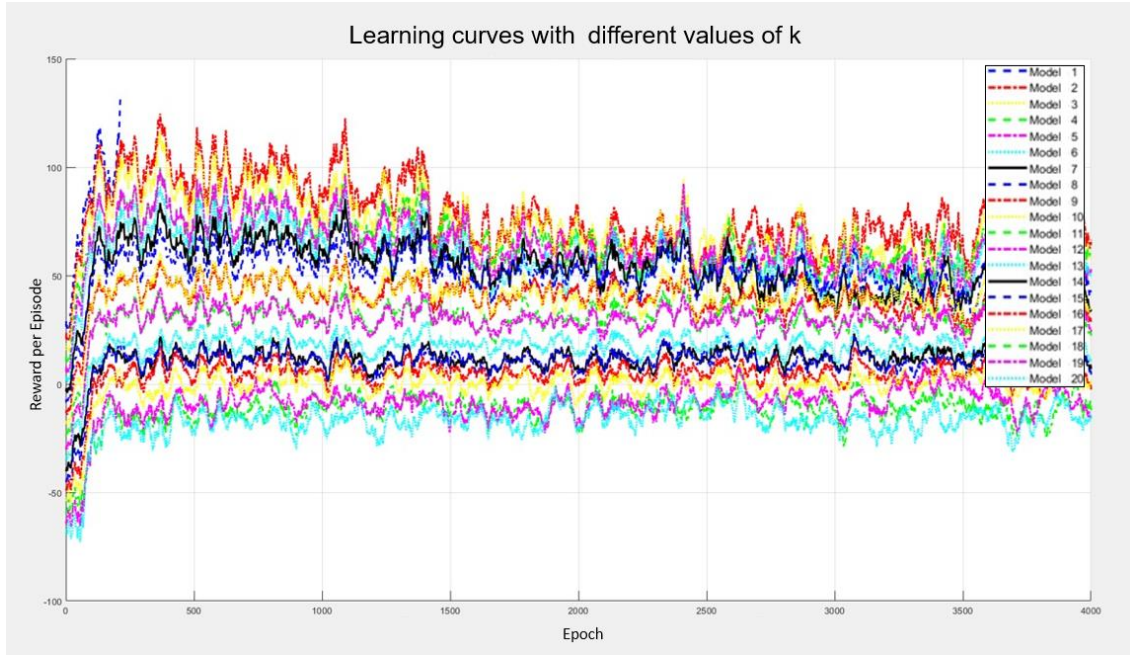


Figure 3.1.1. Impact of k Calibration on Agent Training

Subsequently, tests were conducted in simulation using datasets from different users to evaluate the model's ability to generalize and to analyze the differences in results depending on the dataset. In Figure 3.1.2, the learning curves for different users are visualized. A moving average with a window of 30 is applied to smooth the curves.

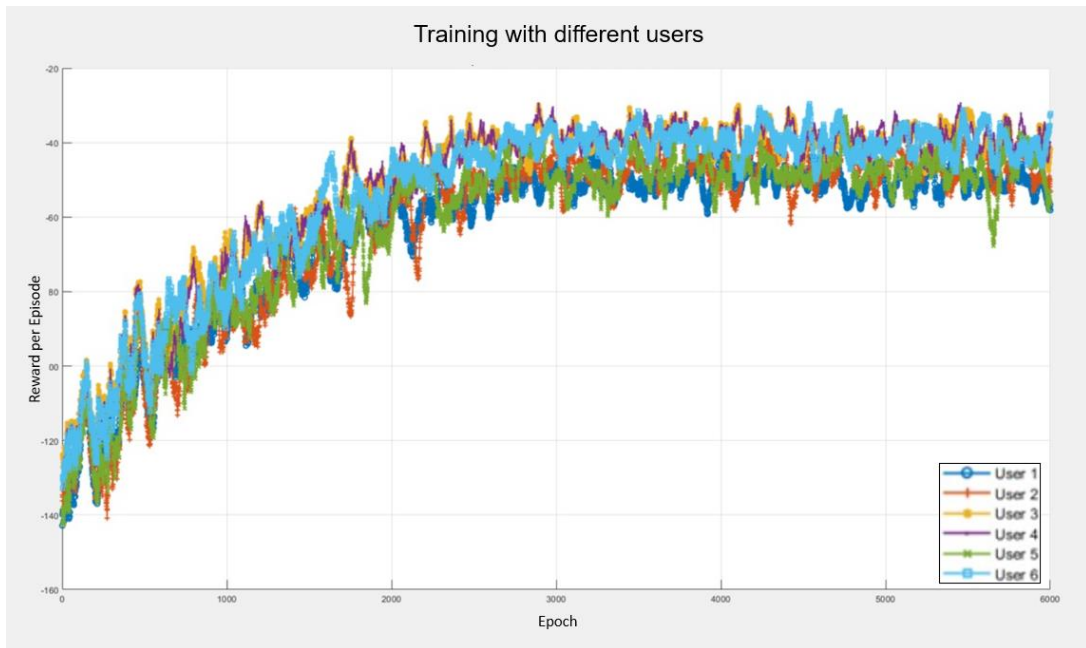


Figure 3.1.2. Learning curves of different datasets.

As can be observed, the learning curves do not differ significantly from each other, which implies that the results obtained for the datasets from different users are quite similar. Figure 3.1.3 shows the graphs of the episodes at the beginning and end of the training, allowing us to appreciate the learning obtained throughout the episodes.

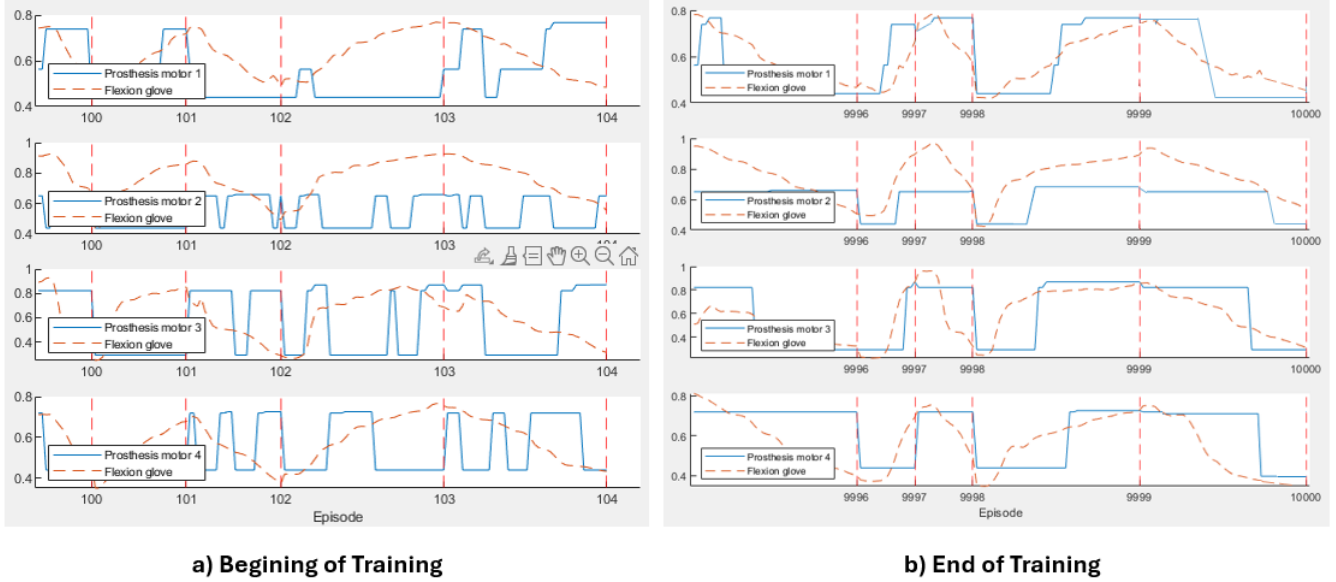


Figure 3.1.3. Episode graphs.

Another test performed was training with the complete dataset to see the variation of training with partial datasets versus the one containing information from all 12 people. In Figure 3.1.4, the learning curve for the complete dataset is shown, maintaining the same configurations as in the partial trainings.

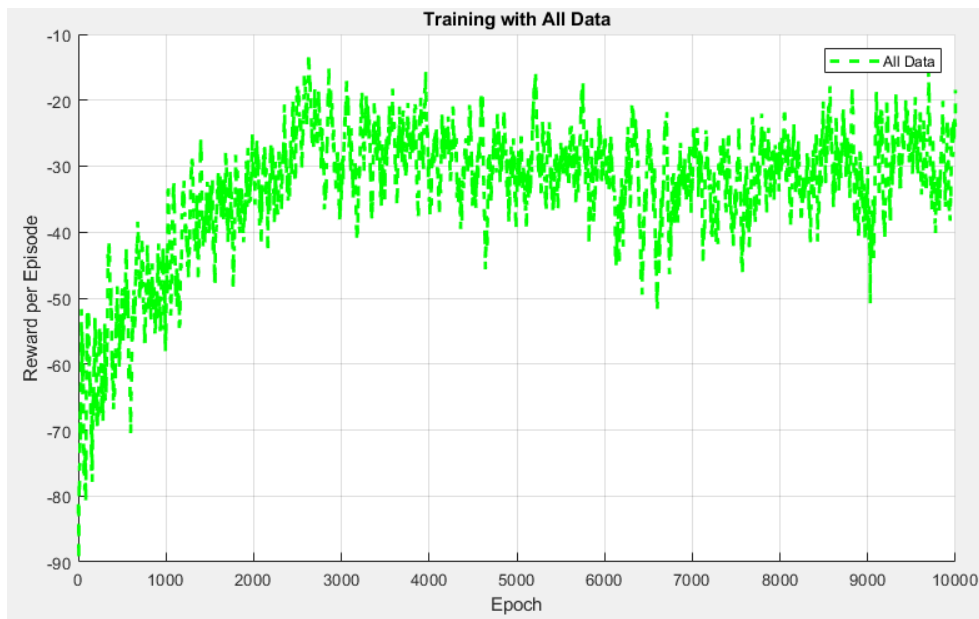


Figure 3.1.4. Training for the complete dataset.

Similarly, the training was analyzed throughout the episodes to verify if similar results are obtained when training with part of the dataset and with the complete dataset. In Figure 3.1.5, the variations in learning at the beginning and end of the training are observed.

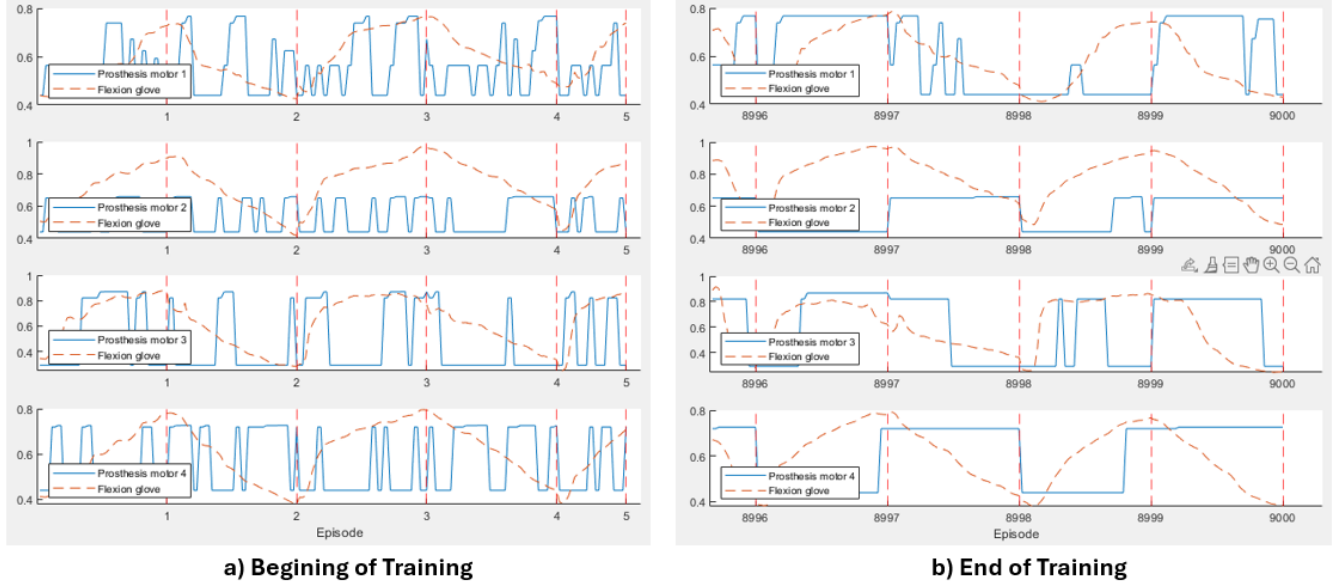


Figure 3.1.5. Episode graphs for the complete dataset.

Based on the results obtained, it was decided to train with a single user from the dataset until the best results were obtained and then train with the complete dataset. Table 3.1.1 details the architecture and hyperparameters used to obtain the best results.

Table 3.1.1. Configurations for training in simulation.

Configuration	α	λ	β	μ	γ	τ	ν	ϵ_{decay}	ϵ_{min}
F	1E-03	1E-04	32	5000	0.99	2	1E-04	5E-05	0.15

Once the model was trained, the agent was tested in simulation to observe its ability to replicate the flexion and extension signals. For this, testing of 50 episodes was performed as shown in Figure 3.1.6.

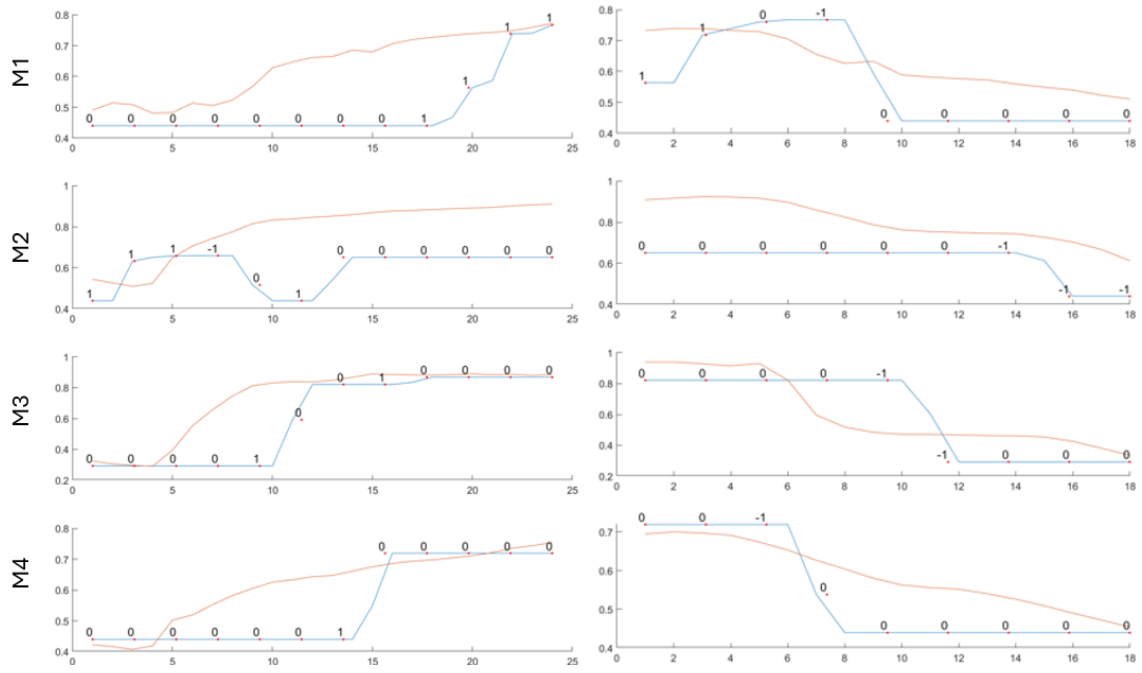


Figure 3.1.6. Testing of the trained agent.

As can be seen, the results in some episodes are as desired, but in others, it can be observed that the glove signal is very far from the motor signals, which represent the agent's prediction. Figure 3.1.7 shows the behavior of the motors throughout all episodes.

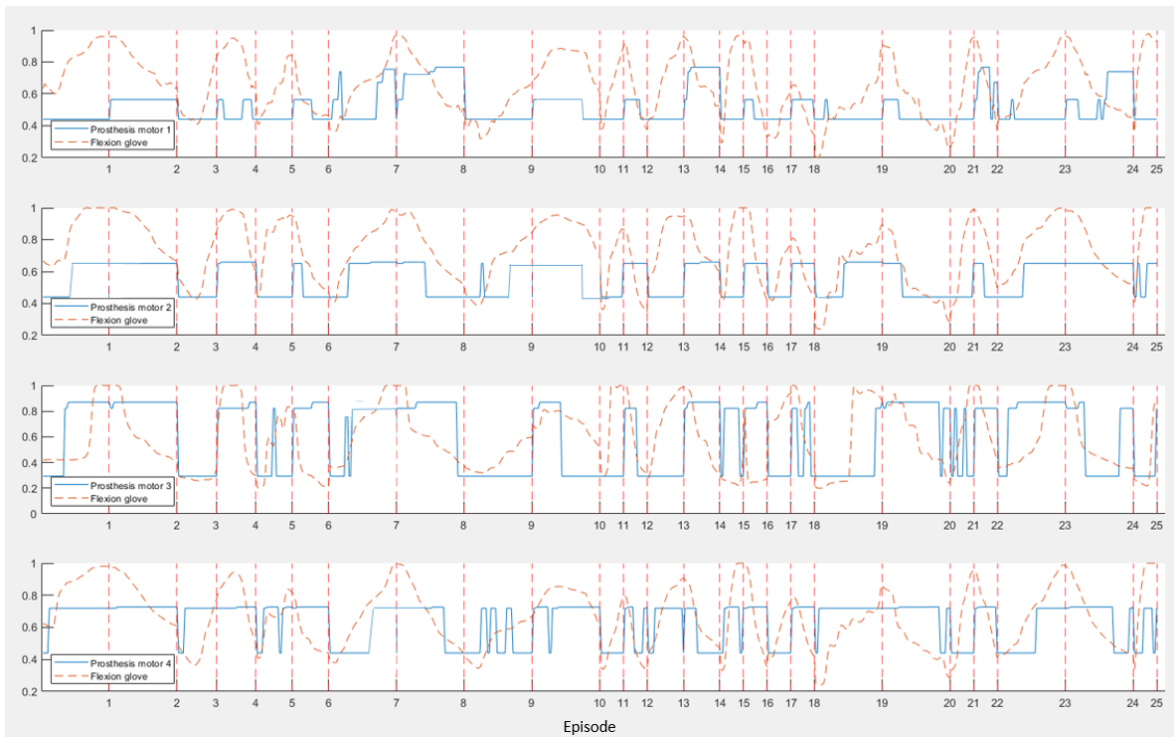


Figure 3.1.7. Motor behavior history during testing.

With the best agent, the fine-tuning process was carried out by transferring the training to hardware. In other words, the simulator and dataset were no longer used; instead, the prosthesis, Myo Armband, and flexion glove were employed. This allowed the agent's learning to be adjusted to real-life conditions. Table 3.1.2 specifies the changes in hyperparameters for fine-tuning, and some tests were carried out with these adjustments to obtain the best results, as shown in Figure 3.1.8.

Table 3.1.2. Tests with different hyperparameters for fine-tuning.

α	λ	β	μ	γ
1E-03	1E-04	1024	10000	0.99
1E-04	1E-05	512	20000	0.99
1E-05	1E-03	2048	15000	0.95

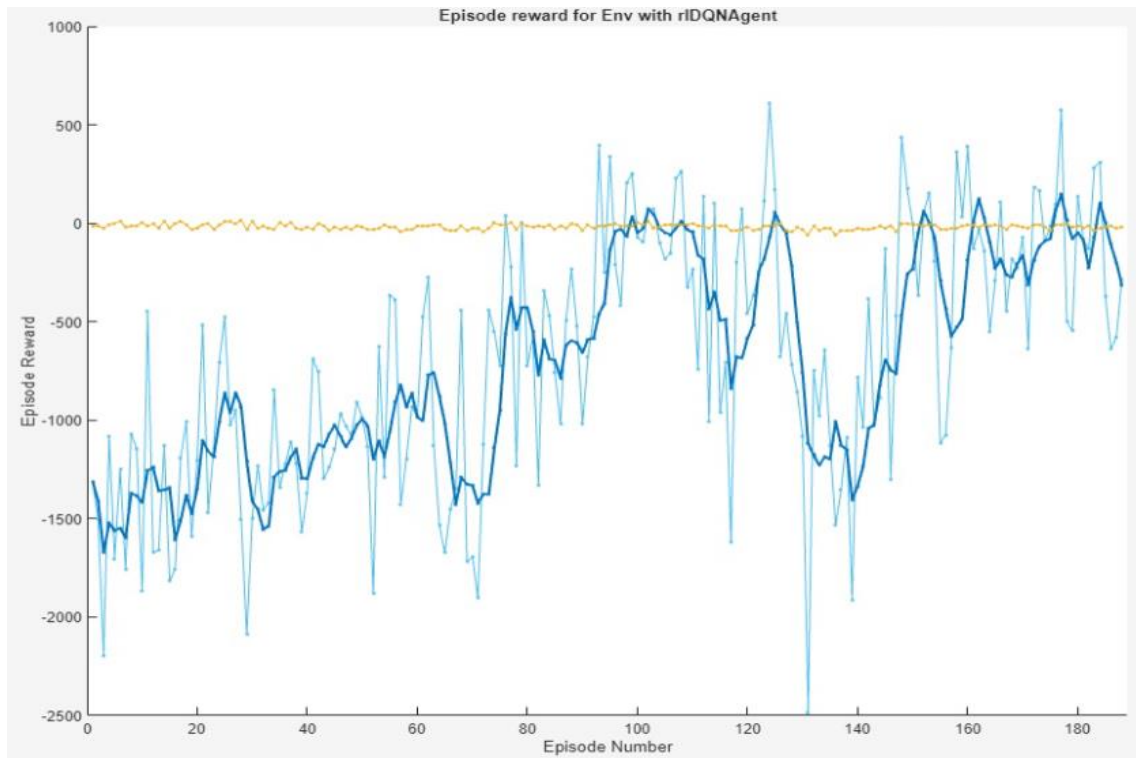


Figure 3.1.8. Training after fine-tuning.

Once the fine-tuning was completed, the agent was tested again to observe its ability to replicate the flexion and extension events, as shown in Figure 3.1.9.

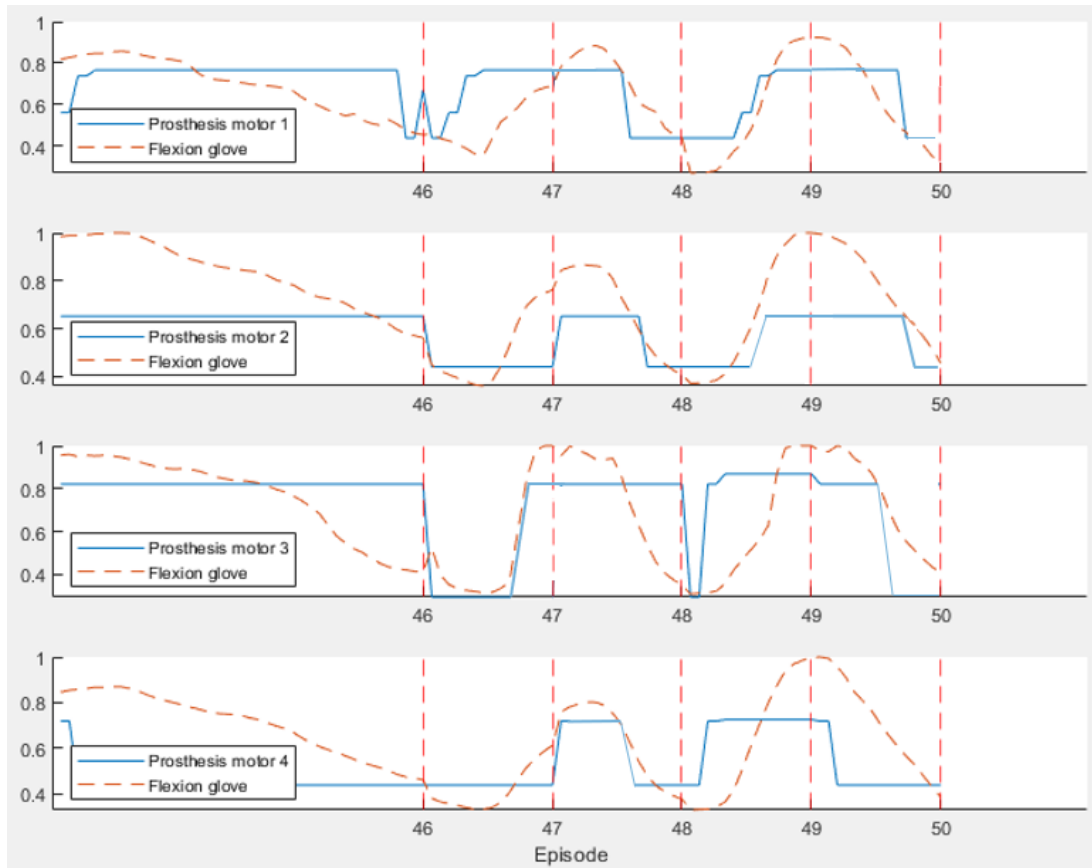


Figure 3.1.9. Testing of the agent after fine-tuning.

Finally, the agent that achieved the best results in the simulation tests was directly tested on the hardware, specifically the myoelectric prosthesis. Electromyographic (EMG) signals generated by the user's residual muscles were captured so that the agent could control the prosthesis actuators, allowing it to accurately replicate the actions performed by the user's real hand. During the tests, various motor tasks were evaluated, such as opening and closing the hand, as shown in Figure 3.1.10.

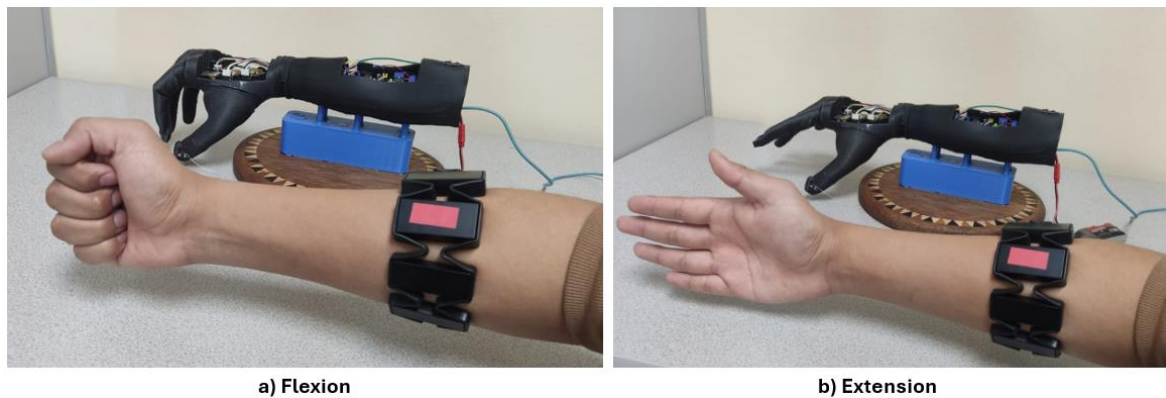


Figure 3.1.10. Testing of the agent on prosthesis.

The source code and other resources used in the development of this work are available in the following GitHub repository: https://github.com/laboratorioAI/EMG_Prosthesis_DQN. Additionally, a video demonstrating the functionality of the prosthesis can be viewed at: <https://youtu.be/K1C-eG7tR8I>.

3.2 Conclusions

This work introduces an innovative approach by applying advanced deep learning techniques, specifically DQL, to improve the interpretation of electromyographic (EMG) signals in myoelectric prostheses. Despite obtaining acceptable results in training and testing in simulation and subsequently in hardware, challenges were encountered in ensuring that the prosthesis could replicate extension and flexion movements performed by the user.

The review of the literature provided a comprehensive understanding of the current state of myoelectric prosthesis control and highlighted the potential of applying Deep-Q Learning (DQL) algorithms. This foundational knowledge informed the design and development of the agent, ensuring that the approach was grounded in the latest advancements and best practices in the field.

Despite the DQL agent showing promising performance in simulations when processing EMG signals and motor position data, during practical tests in a real environment, the prosthesis performed random actions and did not achieve the desired success rate in opening and closing tasks. This can be attributed to several reasons. The inherent variability and noise in EMG signals pose a considerable challenge for the agent, which still requires refinement to interpret and adapt to these signals in real time. Additionally, differences between the simulated environment and real hardware conditions may have contributed to these discrepancies, indicating that continuous calibration and adjustment of the agent in a physical environment require more iteration and specific adjustments that were not fully addressed in this project.

In conclusion, while significant advances were made in the use of DQL for the control of myoelectric prostheses, the results highlight the need to continue improving the precision and robustness of the agent in practical conditions. Future research should focus on adapting and refining the model in real environments, as well as developing techniques to more effectively manage the variability of EMG signals. This will help improve the success rate in executing opening and closing tasks, bringing it closer to replicating the natural movements of the human hand.

3.3 Recommendations

It is essential to improve the quality and consistency of the collected EMG data. This can be achieved by using higher precision sensors and implementing advanced signal filtering techniques to reduce noise and variability.

Expanding the group of participants to include a greater variety of physiological characteristics and usage conditions will help train the agent with a more representative dataset, improving its generalization and adaptation to different users.

Implementing a continuous calibration system that adjusts the agent's parameters in real time will allow the agent to better adapt to changes in EMG signals due to factors such as muscle fatigue and electrode displacement.

Developing more realistic simulations that include variabilities and challenges found in real environments will enable the agent to train in more complex situations before being implemented in the hardware.

4 REFERENCES

- [1] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. MIT Press, 2018.
- [2] K. Arulkumaran, M. P. Deisenroth, M. Brundage, and A. A. Bharath, "A Brief Survey of Deep Reinforcement Learning," *IEEE Signal Processing Magazine*, vol. 34, no. 6, pp. 26-38, 2017.
- [3] H. van Hasselt, A. Guez, and D. Silver, "Deep Reinforcement Learning with Double Q-learning," in *Proceedings of the 30th AAAI Conference on Artificial Intelligence*, 2016, pp. 2094-2100.
- [4] M. Atzori, A. Gijssberts, C. Castellini, B. Caputo, A. Mittaz Hager, S. Elsig, G. Giatsidis, F. Bassetto, and H. Müller, "Electromyography data for non-invasive naturally-controlled robotic hand prostheses," *Scientific Data*, vol. 1, no. 1, pp. 1-13, 2014.
- [5] K. Englehart and B. Hudgins, "A robust, real-time control scheme for multifunction myoelectric control," *IEEE Transactions on Biomedical Engineering*, vol. 50, no. 7, pp. 848-854, 2003.
- [6] M. Kim, A. M. Simon, and L. J. Hargrove, "Seamless and intuitive control of a powered prosthetic leg using deep neural network for transfemoral amputees," *Wearable Technologies*, vol. 3, pp. e24, 2022.
- [7] B. Laschowski, W. McNally, A. Wong, and J. McPhee, "Environment classification for robotic leg prostheses and exoskeletons using deep convolutional neural networks," *Frontiers in Neurorobotics*, vol. 15, p. 730965, 2022.
- [8] Y. Hutabarat, K. Ekkachai, M. Hayashibe, and W. Kongprawechanon, "Reinforcement Q-Learning Control With Reward Shaping Function for Swing Phase Control in a Semi-active Prosthetic Knee," *Frontiers in Neurorobotics*, vol. 14, p. 565702, 2020.
- [9] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," *International Conference on Learning Representations (ICLR)*, 2016.
- [10] S. Gu, E. Holly, T. Lillicrap, and S. Levine, "Deep Reinforcement Learning for Robotic Manipulation with Asynchronous Off-Policy Updates," *International Conference on Robotics and Automation (ICRA)*, 2017.

- [11] D. J. Felleman, A. T. Bolger, A. C. Sanderson, "Robust Design and Testing of a Prosthetic Hand Using Electromyographic Signals," *Nature Preprints*, 2012.
- [12] L. Resnik, H. Huang, A. Winslow, D. L. Crouch, F. Zhang, and N. Wolk, "Evaluation of EMG pattern recognition for upper limb prosthesis control: a case study in comparison with direct myoelectric control," *Journal of NeuroEngineering and Rehabilitation*, vol. 15, no. 23, pp. 1-13, 2018.
- [13] E. Scheme y K. Englehart, "Electromyogram pattern recognition for control of powered upper-limb prostheses: State of the art and challenges for clinical use," *Journal of Rehabilitation Research & Development*, vol. 48, no. 6, pp. 643-660, 2011. DOI: 10.1682/JRRD.2010.09.0177.